

HOSPITAL MANAGEMENT SYSTEM

Project Output Screenshots, ER Diagram & Relational Database Schema

PostgreSQL DB Fiddle — Project Database

<https://dbfiddle.dev/postgres?checkpoint=1790501331>

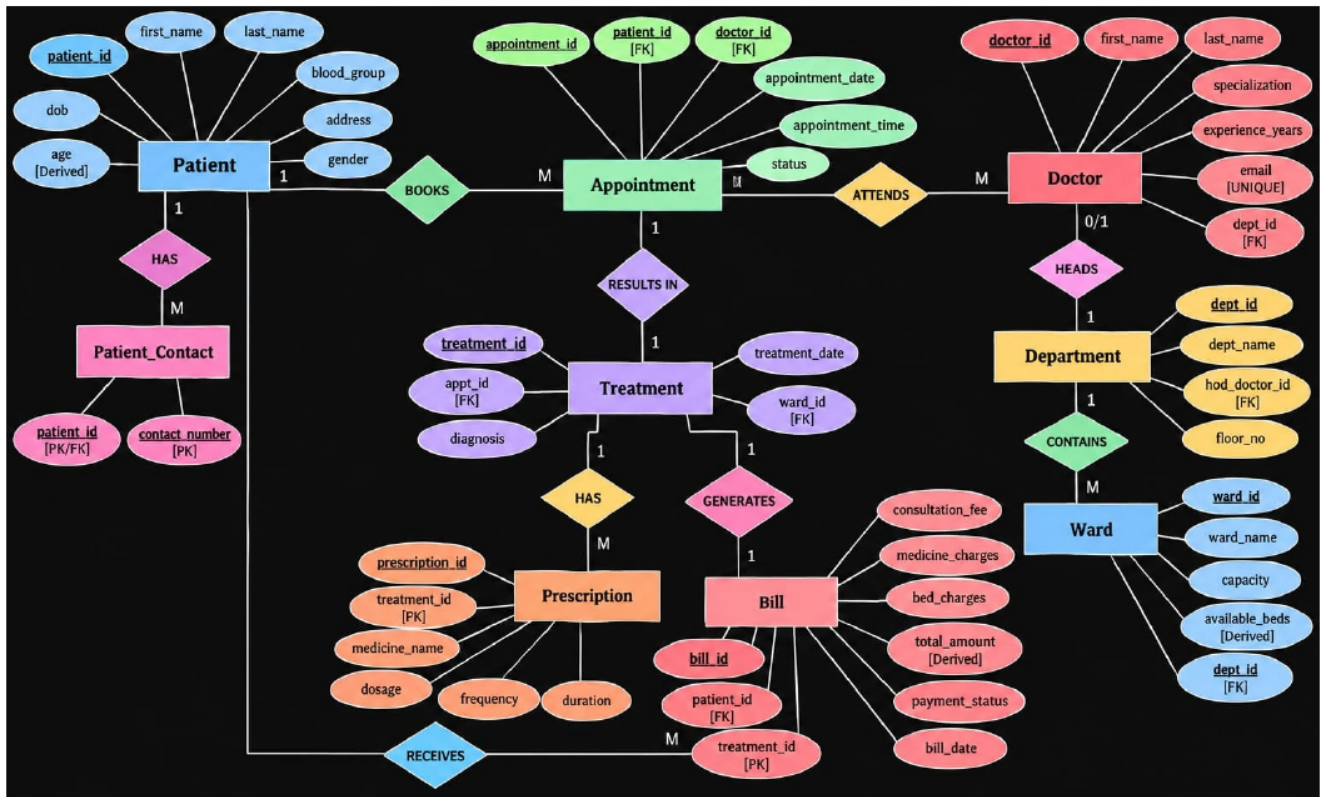
The above link is included as the PostgreSQL DB Fiddle reference supplied for the Hospital Management System project. It is provided as a clickable reference in this PDF; the live Fiddle contents are not reproduced here.

Evidence source: The project figures below were extracted from the previously supplied Hospital Management System final-review presentation. They are included as project evidence and are not newly invented screenshots.

ER Diagram — Hospital Management System



ER DIAGRAM

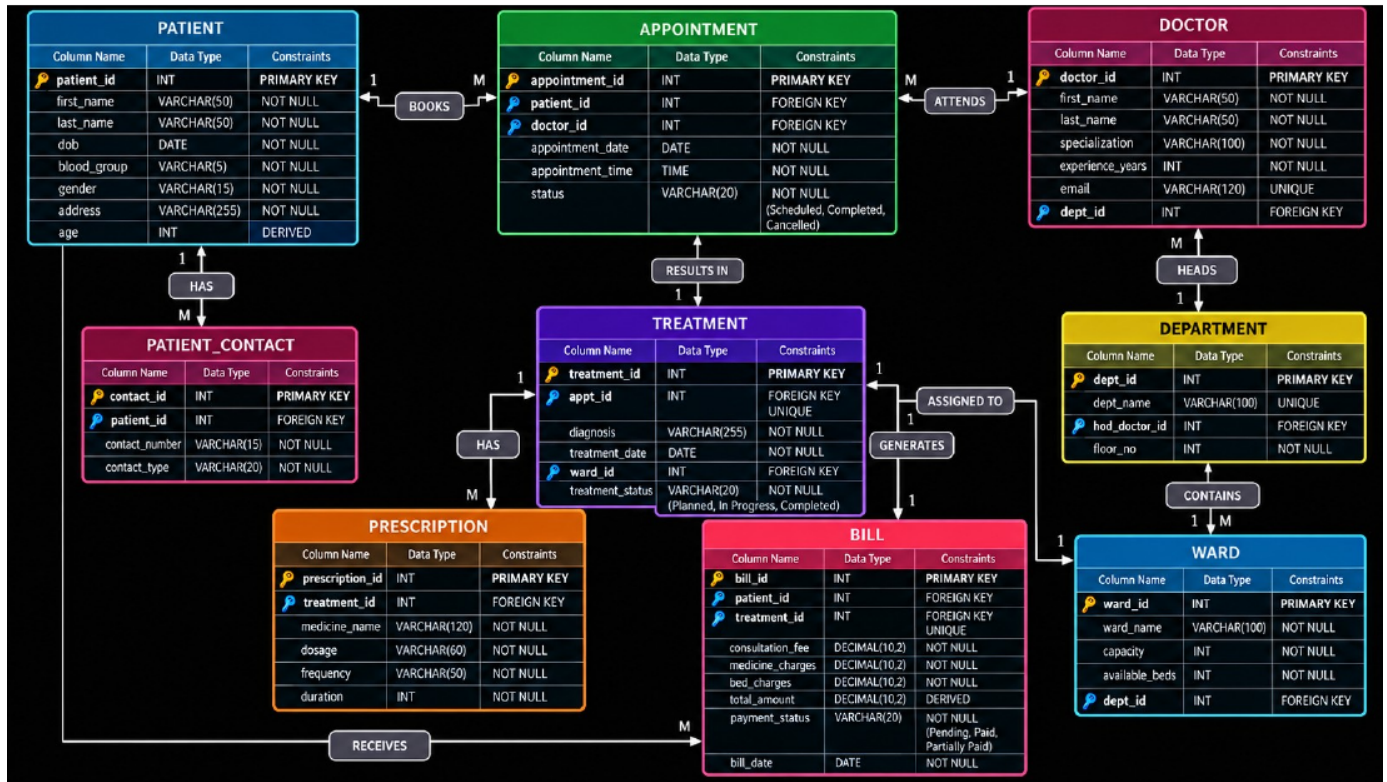


Source: Hospital Management System final-review presentation supplied in the project files.

Relational Database Schema — Hospital Management System



RELATIONAL DATABASE SCHEMA



Source: Hospital Management System final-review presentation supplied in the project files.

PostgreSQL DDL/DML/DQL — Patient Table Output



DDL, DML & DQL OPERATIONS

DDL

CREATE • ALTER • DROP •
TRUNCATE

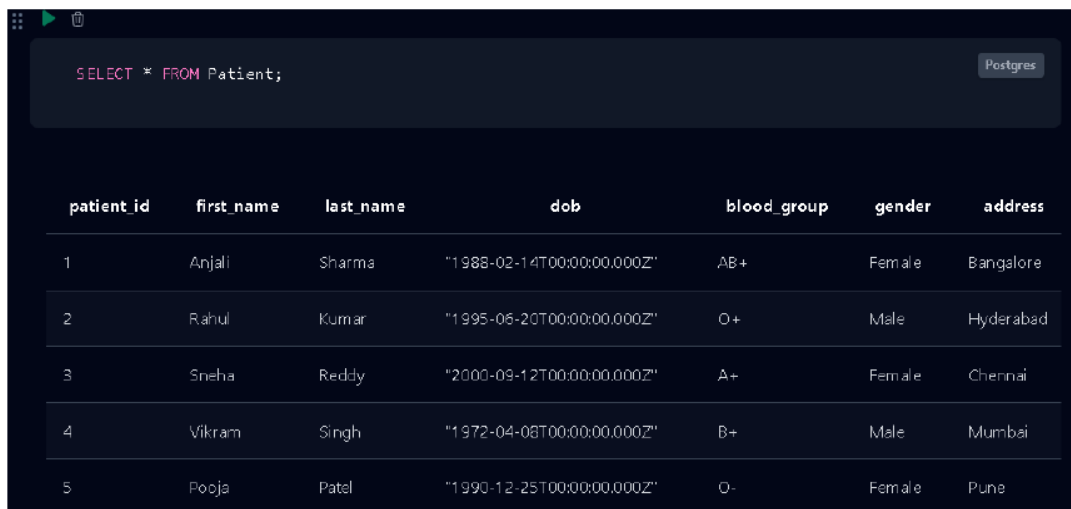
DML

INSERT • UPDATE • DELETE

DQL

SELECT

SELECT * FROM Patient;



```
SELECT * FROM Patient;
```

patient_id	first_name	last_name	dob	blood_group	gender	address
1	Anjali	Sharma	"1988-02-14T00:00:00.000Z"	AB+	Female	Bangalore
2	Rahul	Kumar	"1995-06-20T00:00:00.000Z"	O+	Male	Hyderabad
3	Sneha	Reddy	"2000-09-12T00:00:00.000Z"	A+	Female	Chennai
4	Vikram	Singh	"1972-04-08T00:00:00.000Z"	B+	Male	Mumbai
5	Pooja	Patel	"1990-12-25T00:00:00.000Z"	O-	Female	Pune

Source: Hospital Management System final-review presentation supplied in the project files.

PostgreSQL JOIN / SQL Query Output



CRUD & SQL QUERIES

- CREATE – add a patient or appointment
- READ – retrieve patient and appointment information
- UPDATE – modify patient or appointment details
- DELETE – remove records with referential integrity
- INNER JOIN and LEFT JOIN
- Aggregate Functions
- GROUP BY and HAVING
- Subqueries and Views



The screenshot shows a PostgreSQL query editor with a dark theme. At the top, there's a toolbar with icons for running the query, saving, and undo/redo. The query text is as follows:

```
SELECT p.first_name, d.first_name, a.appointment_date
FROM Patient p
JOIN Appointment a ON p.patient_id = a.patient_id
JOIN Doctor d ON a.doctor_id = d.doctor_id;
```

Below the query, the results are displayed in a table with two columns: **first_name** and **appointment_date**. The table contains five rows of data.

first_name	appointment_date
Rajesh	"2026-09-01T00:00:00.000Z"
Priya	"2026-09-02T00:00:00.000Z"
Arun	"2026-09-03T00:00:00.000Z"
Meena	"2026-09-04T00:00:00.000Z"
Kiran	"2026-09-05T00:00:00.000Z"

Source: Hospital Management System final-review presentation supplied in the project files.

PostgreSQL Data Integrity / CHECK Constraint Validation



DATA INTEGRITY & VALIDATION

The database maintains accuracy and consistency using constraints and validation rules.

CONSTRAINTS USED

- **PRIMARY KEY** prevents duplicate IDs
- **FOREIGN KEY** prevents invalid references
- **NOT NULL** ensures required values are provided
- **UNIQUE** prevents duplicate email/contact values
- **CHECK** validates permitted values

VALIDATION RESULTS

- Duplicate Patient ID → **Rejected**
- Invalid Patient/Doctor Foreign Key → **Rejected**
- Missing required value → **Rejected**
- Duplicate Doctor Email → **Rejected**
- Invalid CHECK value → **Rejected**

```
INSERT INTO Doctor
(first_name, last_name, specialization, experience_years, email, dept_id)
VALUES
('Test', 'Doctor', 'Cardiology', -5,
'test.doctor@hospital.com', 1);
```

error: new row for relation "doctor" violates check constraint "doctor_experience_years_check"

Source: Hospital Management System final-review presentation supplied in the project files.

PostgreSQL Database Reference

DB Fiddle: <https://dbfiddle.dev/postgres?checkpoint=1790501331>

Use this link during the project demonstration to open the PostgreSQL database environment. For the final submission, the SQL implementation, screenshots, diagrams, and live database should be kept consistent with the actual PostgreSQL schema and outputs.